

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikroprosesor

Valian Tsaqif Hidayat - 5024241070

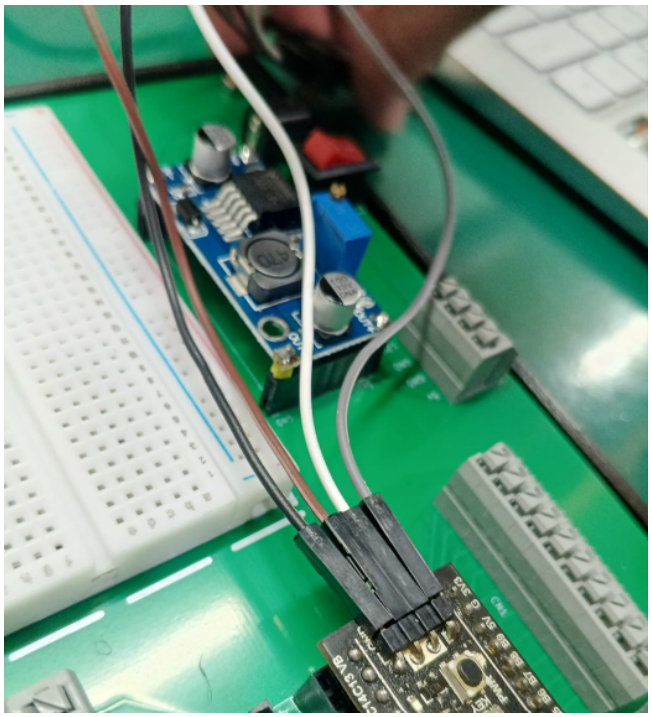
2026

1 Langkah-Langkah Percobaan

1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

1. Pastikan project Modul 1 sudah dibuat dan isi platformio.ini berisi environment blackpill_f401cc dengan platform ststm32, board blackpill_f401cc, framework arduino, serta upload protocol dan debug tool stlink. Tunggu PlatformIO menyelesaikan proses refresh konfigurasi sebelum menekan Build atau Upload.

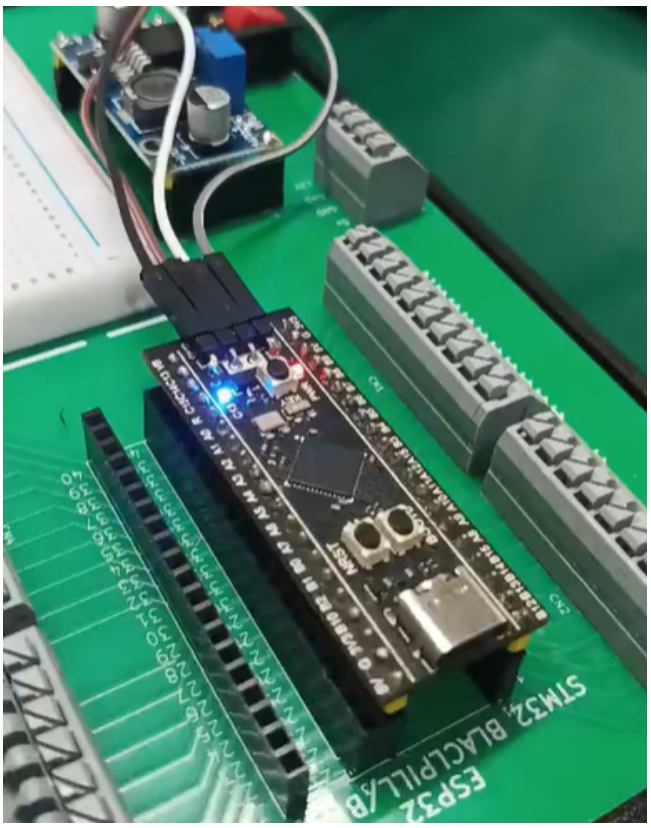
2. Hubungkan ST-Link ke Blackpill dengan kabel. Sambungkan pin SWDIO ke SWDIO, SWCLK ke SWCLK, GND ke GND, dan 3.3V ke 3V3. Pastikan ST-Link memakai tegangan 3.3 V. Colokkan ST-Link ke komputer melalui USB. Periksa nama pin pada ST-Link karena letak pin berbeda pada tiap perangkat.



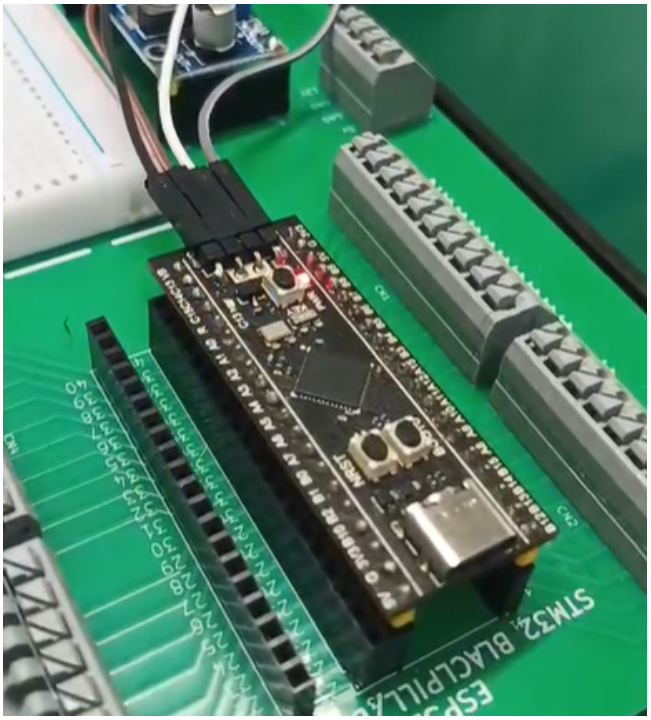
Gambar 1: PIN Tersambung

3. Tulis kode Blink pada src/main.cpp dengan LED onboard pada pin PC13. Atur pin sebagai output pada fungsi setup(). Pada fungsi loop(), tulis LOW, tunggu 500 ms, tulis HIGH, lalu tunggu 500 ms. Klik Build, lalu klik Upload.

4. Amati LED onboard. LED menyala selama satu detik pertama, mati selama satu detik berikutnya, dan berulang terus.



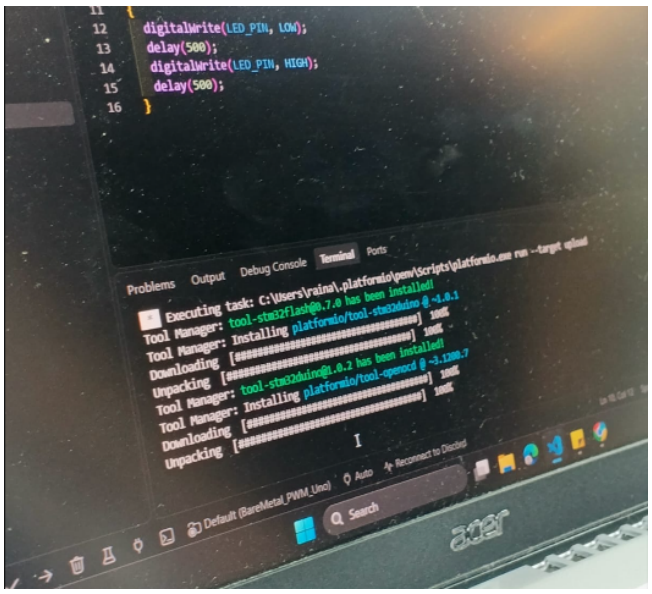
Gambar 2: Nyala



Gambar 3: Mati

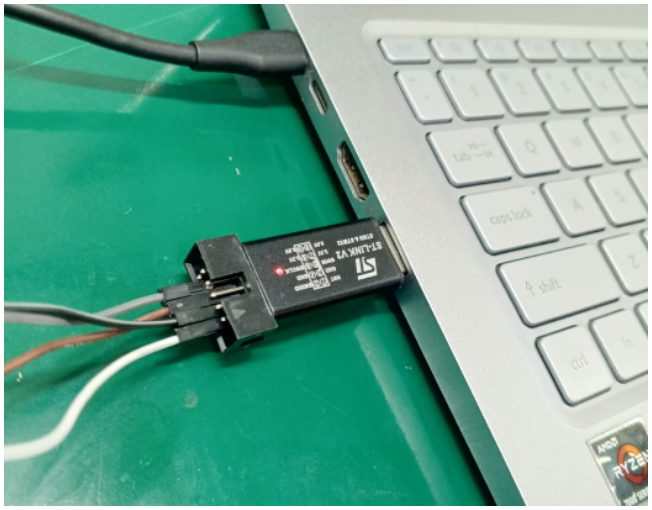
1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

1. Ganti isi platformio.ini dengan environment esp32dev, platform espressif32, board esp32dev, dan framework espidf. Tunggu proses refresh konfigurasi selesai.



Gambar 4: Konfigurasi

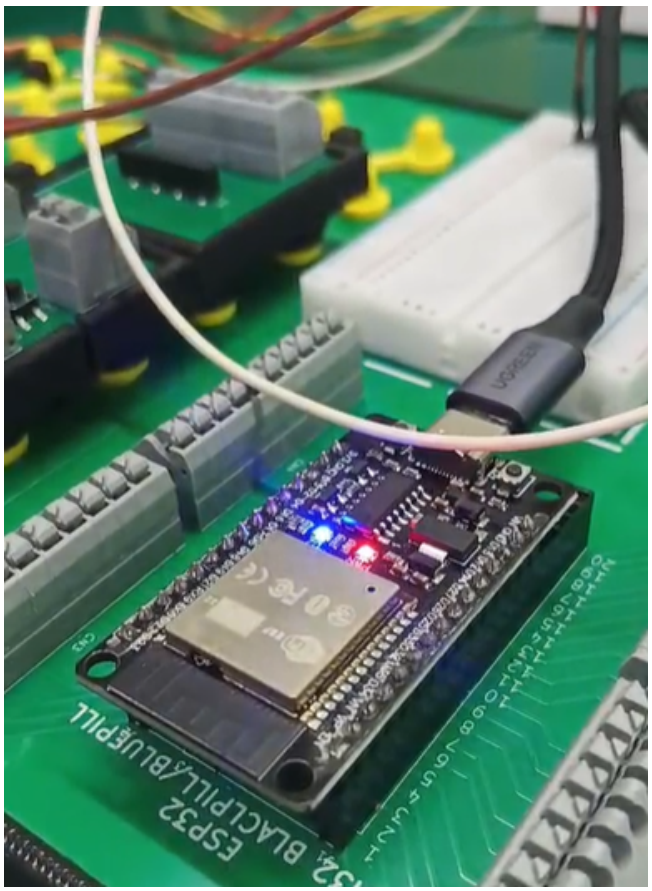
2. Colokkan ESP32 ke komputer melalui USB dan pastikan komputer mengenali port ESP32.



Gambar 5: ESP32 Terhubung ke Laptop melalui USB

3. Hapus file src/main.cpp dan buat file baru src/main.c. Tulis kode Blink yang memuat header FreeRTOS, task, dan driver GPIO. Definisikan LED pada GPIO_NUM_2. Pada fungsi app_main(), reset pin, atur arah pin sebagai output, lalu ulangi perintah set level 1, delay 500 ms, set level 0, dan delay 500 ms di dalam loop while.

4. Klik Build, lalu klik Upload. Amati LED onboard.



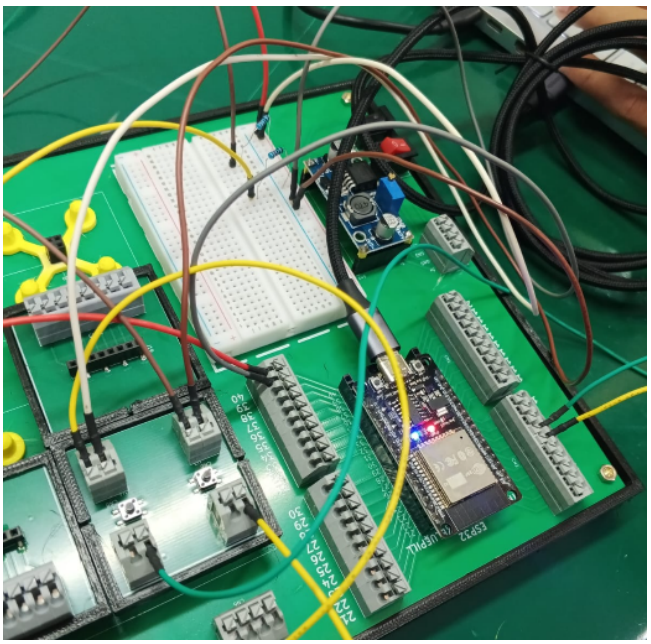
Gambar 6: LED

5. Bandingkan cara memprogram dengan ESP-IDF terhadap Arduino. Catat perbedaan pada titik masuk program, fungsi pengaturan pin, dan fungsi delay.

1.3 Percobaan 3: Pull-up dan Pull-down, Eksternal vs Internal

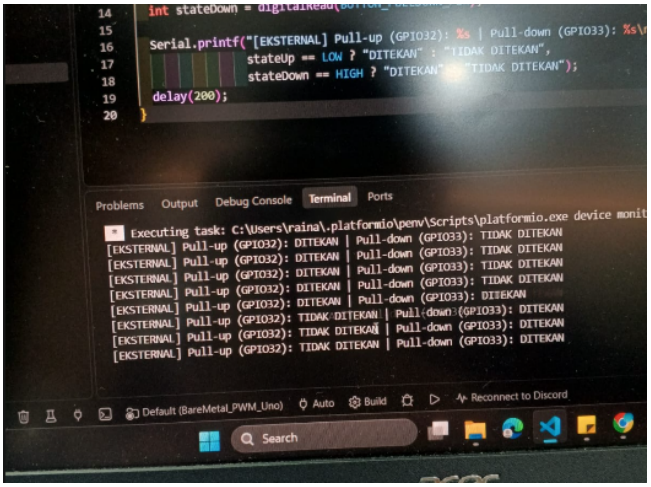
1. Ganti isi platformio.ini kembali ke konfigurasi framework Arduino untuk board esp32dev. Hapus src/main.c, lalu buat file baru src/main.cpp.

2. Pasang dua tombol pada GPIO 32 dan GPIO 33 lengkap dengan resistornya sesuai Skema Bagian A. Pushbutton 1 memakai pull-up eksternal. Satu kaki tombol terhubung ke GPIO 32 dan ke 3.3V melalui resistor 10 kΩ, kaki lain terhubung ke GND. Pushbutton 2 memakai pull-down eksternal. Satu kaki tombol terhubung ke GPIO 33 dan ke GND melalui resistor 10 kΩ, kaki lain terhubung ke 3.3V.



Gambar 7: Rangkaian Sesuai Skema Bagian A

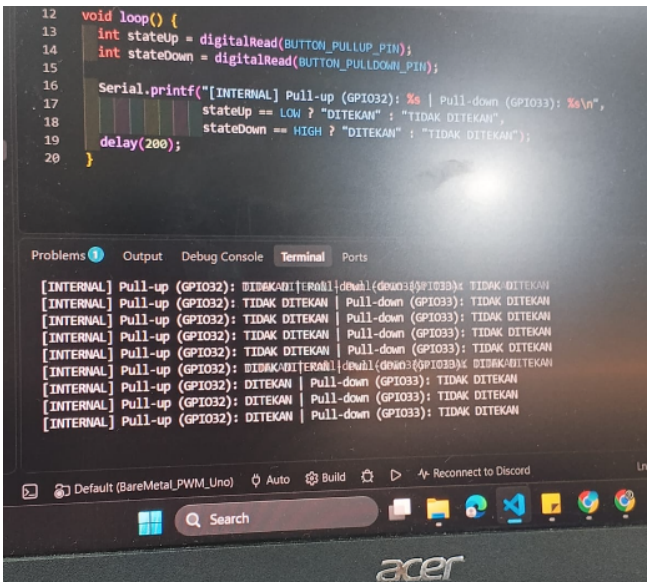
3. Tulis Kode Program A pada src/main.cpp. Kode ini mengatur kedua pin sebagai INPUT biasa dan mencetak status tombol ke Serial Monitor setiap 200 ms. Klik Build dan Upload, lalu tekan kedua tombol dan catat hasil dari Serial Monitor.



Gambar 8: Hasil ketika Button Ditekan Setelah Build dan Upload Kode Program A

4. Lepas kedua resistor sesuai Skema Bagian B. Tombol tetap terpasang. Perhatikan orientasi kaki tombol tactile empat kaki pada Gambar 8 dan Gambar 9 agar pasangan kaki yang dipakai benar.

5. Ganti isi src/main.cpp dengan Kode Program B. Kode ini mengatur GPIO 32 sebagai INPUT_PULLUP dan GPIO 33 sebagai INPUT_PULLDOWN. Klik Build dan Upload, lalu tekan kedua tombol dan catat hasil dari Serial Monitor.



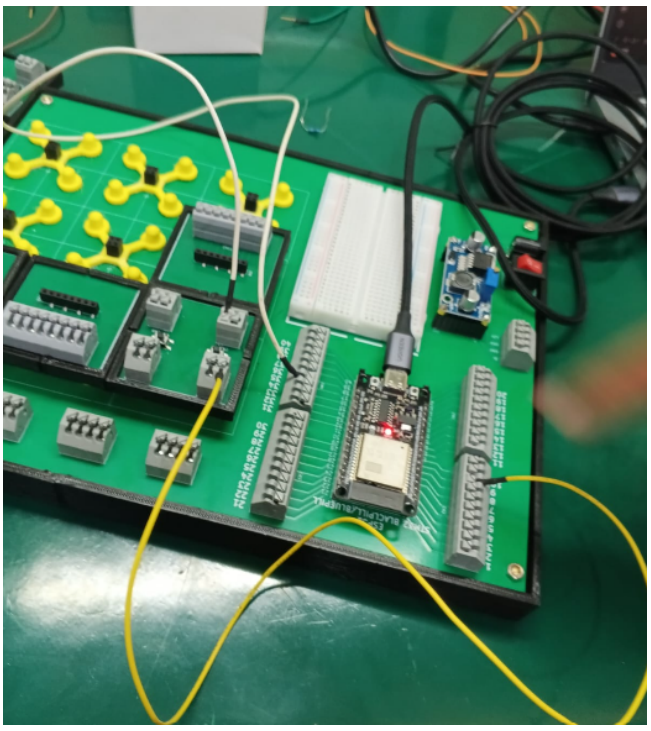
Gambar 9: Ketika Button Ditekan Program B

6. Bandingkan hasil Kode A dan Kode B. Hasil seharusnya sama, yaitu pull-up bernilai HIGH saat tombol tidak ditekan dan LOW saat ditekan, sedangkan pull-down bernilai sebaliknya.

7. Jika hasil terbalik atau berbeda, periksa kembali pemasangan resistor dan pasangan kaki tombol.

1.4 Percobaan 4: Debouncing pada Input Tombol GPIO

1. Pasang tombol sesuai skema. Satu kaki tombol terhubung ke GPIO 25 dan kaki lain terhubung ke GND. Gunakan INPUT_PULLUP. Konfigurasi platformio.ini sama dengan Percobaan 3, sehingga langkah refresh dapat dilewati.

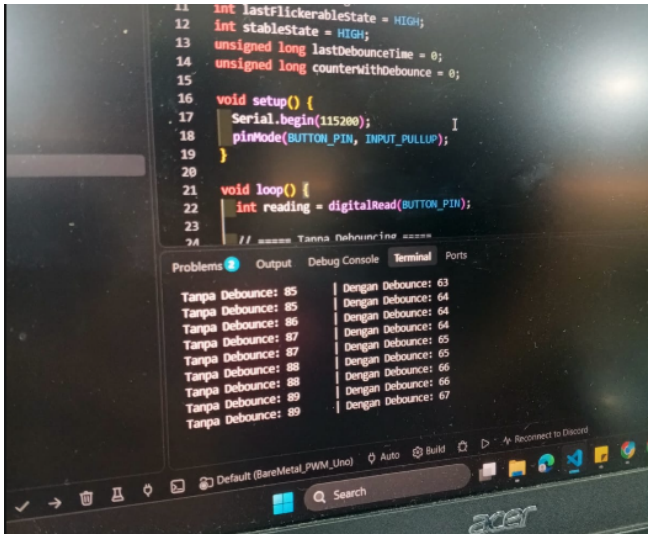


Gambar 10: Rangkaian Sesuai Skema Percobaan 4

2. Tulis kode perbandingan pada src/main.cpp. Kode memiliki dua penghitung, yaitu counterNo-Debounce tanpa penyaring getaran dan counterWithDebounce dengan penyaring getaran. Atur DEBOUNCE_DELAY sebesar 50 ms.

3. Klik Build dan Upload, lalu buka Serial Monitor.

4. Tekan tombol satu kali dan bandingkan nilai kedua penghitung pada Serial Monitor.



Gambar 11: Bandingkan kedua nilai

5. Ulangi penekanan dengan cara berbeda, yaitu cepat dan lambat. Nilai counterNoDebounce bertambah lebih dari satu untuk satu tekanan, sedangkan counterWithDebounce naik tepat satu. Jika kedua nilai terlihat sama, tekan tombol lebih lambat atau lebih cepat karena getaran tidak selalu muncul.

6. Ubah DEBOUNCE_DELAY menjadi 10 ms, lakukan Build dan Upload, lalu tekan tombol lagi.

7. Ubah DEBOUNCE_DELAY menjadi 200 ms, lakukan Build dan Upload, lalu tekan tombol dengan cepat berulang kali.

8. Bandingkan hasil untuk nilai 10 ms, 50 ms, dan 200 ms. Catat jumlah hitungan yang muncul dan respons tombol pada tiap nilai.

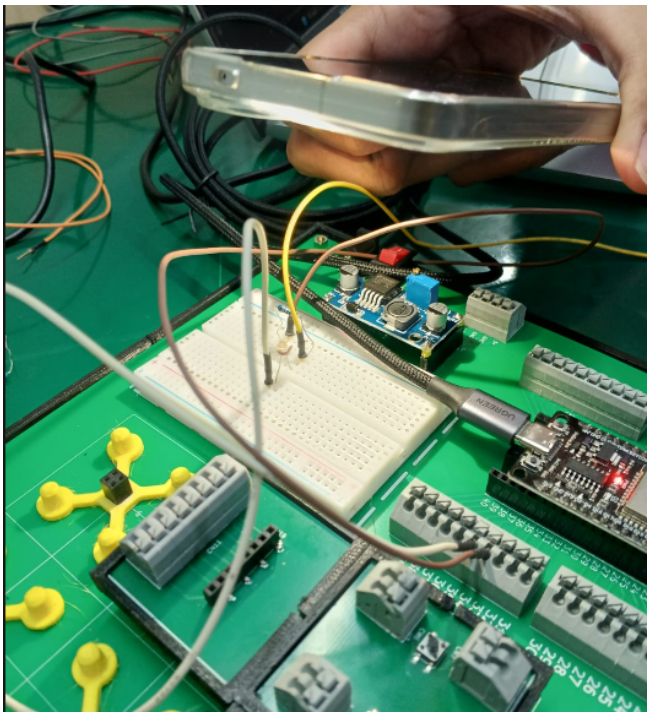
1.5 Percobaan 5: Akses GPIO Analog, Pembacaan ADC dengan LDR

1. Pasang LDR dan resistor 10 kΩ sesuai skema pembagi tegangan. Satu kaki LDR terhubung ke 3.3V. Kaki lain LDR terhubung ke GPIO 34 dan ke GND melalui resistor 10 kΩ. Titik tengah kedua komponen menjadi masukan ADC.

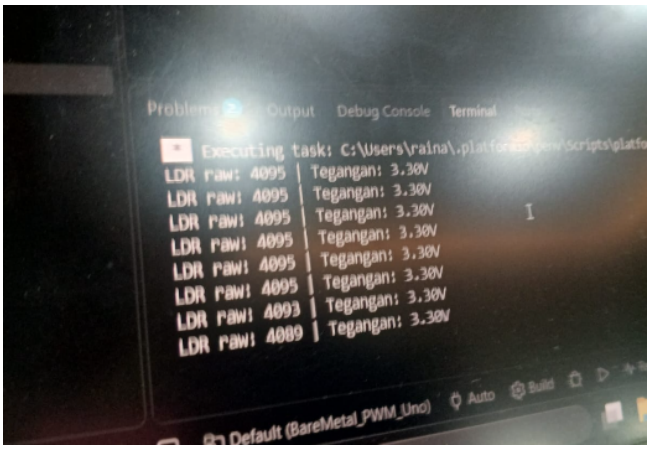
2. Tulis program pada src/main.cpp yang membaca nilai ADC mentah dari GPIO 34 dengan fungsi analogRead(), lalu mengubah nilai tersebut menjadi tegangan dengan rumus $V = \text{raw} \times (3{,}3/4095)$. Cetak kedua nilai ke Serial Monitor setiap 300 ms.

3. Klik Build dan Upload, lalu buka Serial Monitor dengan baud rate 115200.

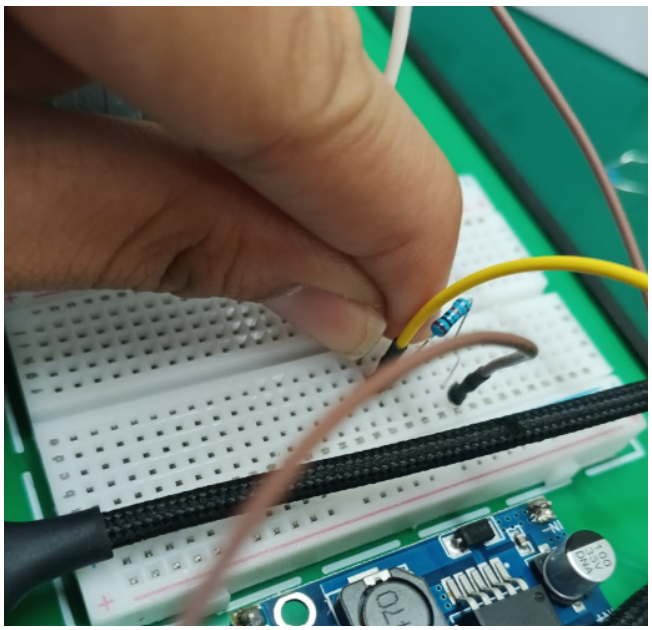
4. Tutup LDR dengan tangan, lalu sinari dengan lampu. Catat nilai mentah pada rentang 0 sampai 4095 untuk setiap kondisi.



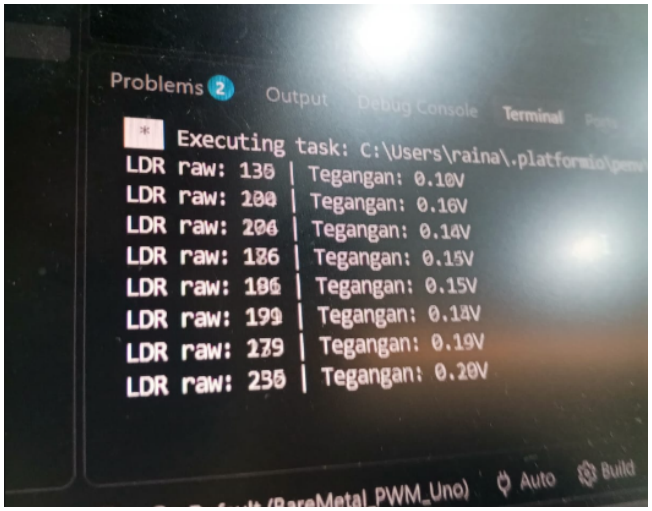
Gambar 12: LDR Disinari Lampu Senter



Gambar 13: Hasil nilai mentah kondisi disinari



Gambar 14: LDR Ditutupi Jari



Gambar 15: Hasil nilai mentah kondisi ditutupi

5. Analisis alasan sinyal analog menunjukkan banyak kondisi, tidak hanya HIGH atau LOW.

2 Analisis Hasil Percobaan

2.1 Analisis Percobaan 1

LED onboard Blackpill menyala selama 500 ms dan mati selama 500 ms secara bergantian. Periode satu siklus bernilai 1 detik, sehingga frekuensi kedipan sebesar 1 Hz. LED pada pin PC13 bersifat aktif LOW. Anoda LED terhubung ke catu positif dan katoda terhubung ke pin PC13. Arus mengalir saat pin bernilai LOW, sehingga perintah LOW menyalakan LED dan perintah HIGH mematikan LED. Hasil ini menunjukkan logika terbalik dibandingkan dugaan umum. Framework Arduino pada STM32 menyederhanakan pemrograman karena label pin seperti PC13 langsung dipakai tanpa konfigurasi struct GPIO_InitTypeDef seperti pada HAL. Fungsi pinMode() mengaktifkan clock peripheral GPIO

secara otomatis. Koneksi ST-Link melalui SWDIO dan SWCLK berfungsi sebagai jalur unggah dan debug. Kegagalan unggah umumnya berasal dari kabel yang longgar, tegangan target yang tidak terbaca, atau proses refresh PlatformIO yang belum selesai.

2.2 Analisis Percobaan 2

LED onboard ESP32 pada GPIO 2 berkedip dengan periode 1 detik, sama seperti hasil pada Percobaan 1. Perbedaan terdapat pada struktur program. Arduino memakai fungsi setup() dan loop() yang menyembunyikan detail sistem. ESP-IDF memakai fungsi app_main() sebagai titik masuk, dan fungsi ini berjalan sebagai task FreeRTOS. Pengaturan pin pada ESP-IDF bersifat eksplisit. Fungsi gpio_reset_pin() dan gpio_set_direction() menggantikan pinMode(), sedangkan gpio_set_level() menggantikan digitalWrite(). Fungsi vTaskDelay(pdMS_TO_TICKS(500)) menunda task berdasarkan tick FreeRTOS. Task lain tetap berjalan selama penundaan, berbeda dengan delay() Arduino yang menahan program utama. Pemrograman dengan ESP-IDF membutuhkan kode lebih panjang dan pemahaman lebih dalam, tetapi memberikan kendali penuh terhadap peripheral dan penjadwalan task. Arduino cocok untuk prototipe cepat. ESP-IDF cocok untuk aplikasi dengan banyak proses bersamaan dan kebutuhan pengaturan tingkat rendah.

2.3 Analisis Percobaan 3

Pada Kode A dan Kode B, tombol dengan pull-up menghasilkan HIGH saat tidak ditekan dan LOW saat ditekan. Pin terhubung ke 3.3V melalui resistor pada kondisi normal, lalu terhubung langsung ke GND saat tombol ditekan. Tombol dengan pull-down menghasilkan LOW saat tidak ditekan dan HIGH saat ditekan. Pin terhubung ke GND melalui resistor pada kondisi normal, lalu terhubung ke 3.3V saat tombol ditekan. Kedua konfigurasi memberi tingkat logika default yang pasti, sehingga pin tidak mengambang. Pin yang mengambang membaca nilai acak akibat derau.

Hasil Kode B sama dengan Kode A walaupun semua resistor eksternal dilepas. ESP32 memiliki resistor pull-up dan pull-down internal sebesar puluhan kiloohm yang aktif melalui INPUT_PULLUP dan INPUT_PULLDOWN. Pemakaian resistor internal mengurangi jumlah komponen dan menyederhanakan rangkaian. Resistor eksternal tetap berguna saat nilai hambatan tertentu dibutuhkan, misalnya pada kabel panjang atau lingkungan dengan derau tinggi, karena nilai 10 kΩ lebih rendah dan lebih tahan gangguan. Hasil terbalik atau tidak konsisten menandakan kesalahan pemasangan, misalnya pasangan kaki tombol yang salah. Kaki tactile pada sisi yang sama selalu terhubung, sehingga pemasangan pada dua kaki di sisi yang sama membuat tombol selalu aktif.

2.4 Analisis Percobaan 4

Tombol mekanik tidak berpindah keadaan secara bersih. Kontak logam memantul beberapa kali dalam beberapa milidetik sebelum stabil, sehingga sinyal berganti HIGH dan LOW berulang kali. Nilai counterNoDebounce naik lebih dari satu untuk satu kali penekanan karena kode menghitung setiap transisi HIGH ke LOW. Nilai counterWithDebounce naik tepat satu karena kode hanya menerima perubahan setelah sinyal stabil selama waktu DEBOUNCE_DELAY. Kode menyimpan waktu perubahan terakhir dengan millis(), lalu memperbarui status stabil bila selisih waktu melewati batas.

Nilai 10 ms terkadang belum cukup. Getaran yang lebih lama dari 10 ms tetap lolos dan menambah hitungan secara keliru. Nilai 50 ms menghasilkan hitungan yang akurat pada sebagian besar

tombol tactile. Nilai 200 ms menghilangkan getaran sepenuhnya, tetapi tombol terasa lambat merespons dan penekanan cepat berulang tidak terhitung karena jeda penerimaan terlalu panjang. Hasil ini menunjukkan adanya kompromi antara kebersihan sinyal dan kecepatan respons. Nilai waktu debounce perlu disesuaikan dengan karakter tombol dan kebutuhan aplikasi.

2.5 Analisis Percobaan 5

Rangkaian LDR dan resistor 10 kΩ bekerja sebagai pembagi tegangan. Tegangan keluaran pada GPIO 34 dinyatakan sebagai

$$V_o = V_{in} \times \frac{R_1}{R + R_1} \tag{1}$$

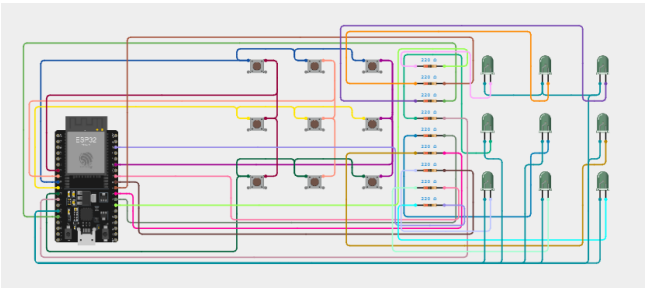
dengan R sebagai hambatan LDR dan R_1 sebagai resistor 10 kΩ. Hambatan LDR turun saat cahaya bertambah. Tegangan V_o naik, sehingga nilai ADC mentah mendekati batas atas 4095 dan tegangan mendekati 3,3 V. Saat LDR ditutup, hambatan LDR naik dan V_o turun, sehingga nilai ADC mentah mendekati 0. Cahaya ruangan biasa menghasilkan nilai di antara kedua batas tersebut.

ADC 12 bit pada ESP32 membagi rentang 0 sampai 3,3 V menjadi 4096 tingkat. Satu tingkat bernilai sekitar 0,8 mV. Sinyal analog menunjukkan banyak kondisi karena nilainya berubah kontinu sesuai intensitas cahaya, sedangkan sinyal digital hanya mengenali dua keadaan, yaitu HIGH dan LOW. Pembacaan dengan GPIO digital hanya menunjukkan apakah tegangan melewati ambang logika dan tidak menunjukkan tingkat terang. GPIO 34 termasuk pin input-only pada ADC1. ADC1 lebih dianjurkan daripada ADC2 karena ADC2 terganggu saat WiFi aktif. ADC ESP32 juga bersifat tidak linear di dekat batas bawah dan batas atas rentang, sehingga nilai mentah di ujung rentang kurang akurat.

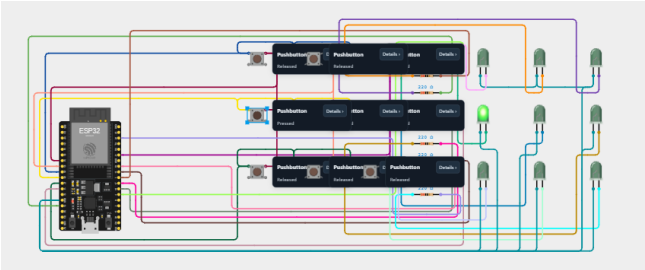
3 Hasil Tugas Modul

Matrix Pushbutton 3x3 Mengendalikan Matrix LED 3x3:

Buatlah simulasi untuk Matrix Pushbutton 3x3 yang mengendalikan Matrix LED 3x3, dimana tombol pada posisi (baris, kolom) tertentu menyalakan LED pada posisi yang sama, mis. tombol baris 1 kolom 1 menyalakan LED baris 1 kolom 1. Program wajib menerapkan debouncing pada setiap tombol (mengacu pada Percobaan 4), sehingga LED hanya berubah status setelah pembacaan tombol benar-benar stabil, bukan langsung mengikuti setiap perubahan sinyal mentah yang masih mungkin bouncing. Berikut merupakan hasil tugas modul:



Gambar 16: Rangkaian Tugas Modul



Gambar 17: Simulasi Ketika Button Ditekan

sketch.ino documentation.txt +

```ino
// 3x3 button matrix: rows driven one at a time LOW; columns use pull-ups.
// Each key has an independent 25 ms stability timer.
const uint8_t rowPins[3] = {25, 26, 27};
const uint8_t colPins[3] = {32, 33, 21};
const uint8_t ledPins[3][3] = {
  {4, 5, 13},
  {14, 16, 17},
  {18, 19, 22}
};
const unsigned long debounceMs = 25;
bool rawPressed[3][3] = {};
bool stablePressed[3][3] = {};
unsigned long lastChangeMs[3][3] = {};

void setup() {
  for (uint8_t r = 0; r < 3; ++r) {
    pinMode(rowPins[r], OUTPUT);
    digitalWrite(rowPins[r], HIGH);
    for (uint8_t c = 0; c < 3; ++c) {
      pinMode(ledPins[r][c], OUTPUT);
      digitalWrite(ledPins[r][c], LOW);
    }
  }
  for (uint8_t c = 0; c < 3; ++c) pinMode(colPins[c], INPUT_PULLUP);
}

void loop() {
  for (uint8_t r = 0; r < 3; ++r) {
    digitalWrite(rowPins[r], LOW);
    delayMicroseconds(100);
    for (uint8_t c = 0; c < 3; ++c) {
      const unsigned long now = millis();
      const bool pressed = (digitalRead(colPins[c]) == LOW);
      if (pressed != rawPressed[r][c]) {
        rawPressed[r][c] = pressed;
        lastChangeMs[r][c] = now;
      }
      if (stablePressed[r][c] != rawPressed[r][c] &&
          (unsigned long)(now - lastChangeMs[r][c]) >= debounceMs) {
        stablePressed[r][c] = rawPressed[r][c];
        digitalWrite(ledPins[r][c], stablePressed[r][c] ? HIGH : LOW);
      }
    }
    digitalWrite(rowPins[r], HIGH);
  }
}
```

Gambar 18: Source Code

3.1 Pertanyaan Analisis

1. Analisis mengapa debouncing tetap diperlukan pada rangkaian matrix pushbutton ini, walaupun pada satu waktu hanya satu tombol yang biasanya ditekan, kaitkan dengan penyebab bouncing pada C.5 Debouncing! Jawaban: Kontak mekanis tombol dapat memantul saat ditekan atau dilepas: alih-alih satu transisi bersih, sinyal sesaat berganti-ganti antara HIGH dan LOW. Mikrokontroler dapat membaca pantulan itu sebagai beberapa perubahan tombol. Meski biasanya hanya satu tombol ditekan, bouncing tetap terjadi pada tombol tersebut; pemindaian baris-kolom tidak menghilangkannya. Debounce menunggu sinyal stabil selama waktu tertentu sebelum memperbarui status LED.

2. Analisis konfigurasi pull-up atau pull-down (sesuai rancangan Anda) yang digunakan pada pembacaan kolom tombol, apa kondisi (HIGH/LOW) pin tersebut saat tombol tidak ditekan, dan mengapa kondisi tersebut tidak boleh dibiarkan floating? Jawaban: Dengan konfigurasi pull-up, input kolom berada pada HIGH ketika tombol tidak ditekan. Saat tombol menghubungkan kolom ke baris yang sedang dipindai pada level LOW, input terbaca LOW. Input tidak boleh dibiarkan floating karena

tanpa jalur pull-up atau pull-down, tegangannya tidak terdefinisi dan dapat terbaca berubah-ubah, sehingga mikrokontroler bisa mendeteksi penekanan palsu.

3. Andaikan salah satu Tombol pada matrix ini digantikan dengan LDR yang dibaca melalui ADC (seperti Percobaan 5), analisis perbedaan mendasar antara membaca GPIO sebagai digital input (tombol, hanya HIGH/LOW) dan sebagai analog input (ADC), mengapa ADC dapat merepresentasikan lebih dari dua kondisi? Jawaban: GPIO digital membaca keadaan logika, umumnya HIGH atau LOW berdasarkan ambang input. ADC mengubah tegangan analog menjadi nilai digital bertingkat; karena nilainya mewakili rentang tegangan, ADC dapat membedakan lebih dari dua tingkat—misalnya intensitas cahaya dari LDR melalui pembagi tegangan. Jumlah tingkat yang dapat dibedakan bergantung pada resolusi ADC.

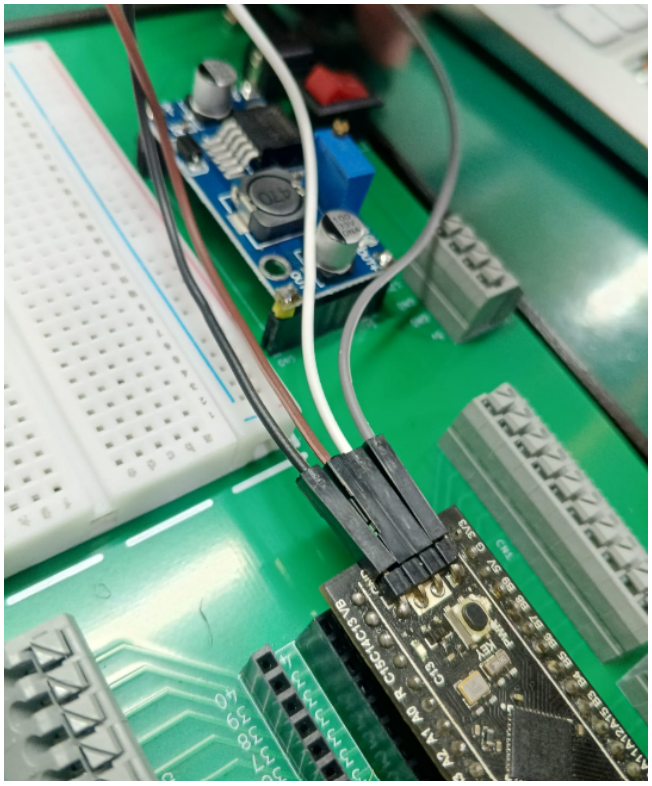
4 Kesimpulan

Praktikum ini memperkenalkan akses GPIO pada STM32 Blackpill dan ESP32 dalam tiga mode, yaitu output digital, input digital, dan input analog. Percobaan 1 menunjukkan kedipan LED berhasil pada STM32 dengan framework Arduino, dan LED onboard PC13 bersifat aktif LOW. Percobaan 2 menunjukkan program Blink pada ESP32 dengan ESP-IDF memakai app_main(), fungsi GPIO eksplisit, dan delay berbasis FreeRTOS. ESP-IDF memberi kendali lebih besar dibandingkan Arduino dengan kode yang lebih panjang.

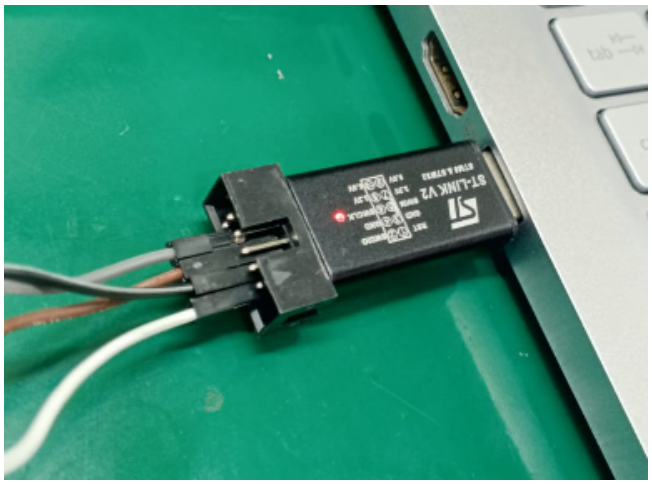
Percobaan 3 menunjukkan resistor pull-up dan pull-down mencegah pin mengambang. Pull-up menghasilkan HIGH saat tombol dilepas dan pull-down menghasilkan LOW saat tombol dilepas. Resistor internal ESP32 menggantikan resistor eksternal dengan hasil yang sama. Percobaan 4 menunjukkan tombol mekanik menghasilkan getaran kontak yang menyebabkan hitungan ganda. Debouncing berbasis software dengan jeda 50 ms menghasilkan satu hitungan untuk satu penekanan. Nilai jeda yang terlalu kecil meloloskan getaran, sedangkan nilai yang terlalu besar memperlambat respons. Percobaan 5 menunjukkan ADC 12 bit membaca tegangan pembagi LDR sebagai nilai 0 sampai 4095 sesuai intensitas cahaya. Sinyal analog memberi informasi tingkat intensitas, sedangkan sinyal digital hanya memberi dua keadaan.

5 Lampiran

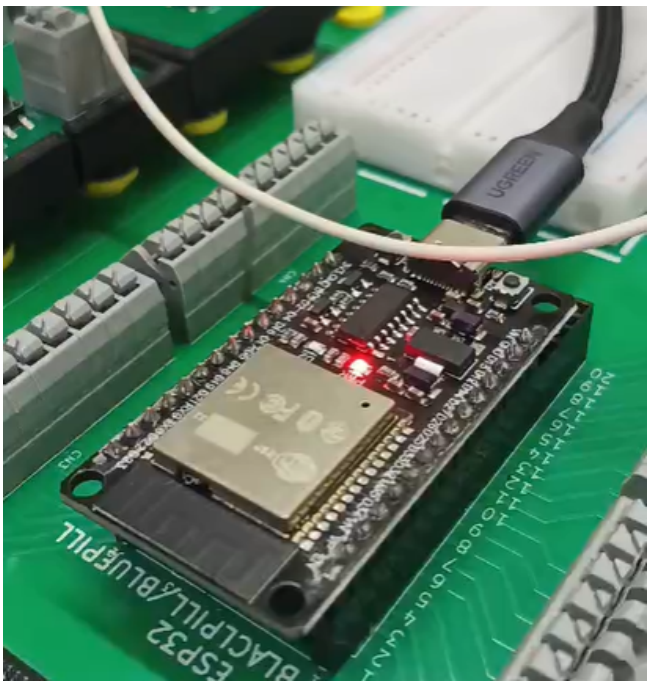
5.1 Dokumentasi saat praktikum



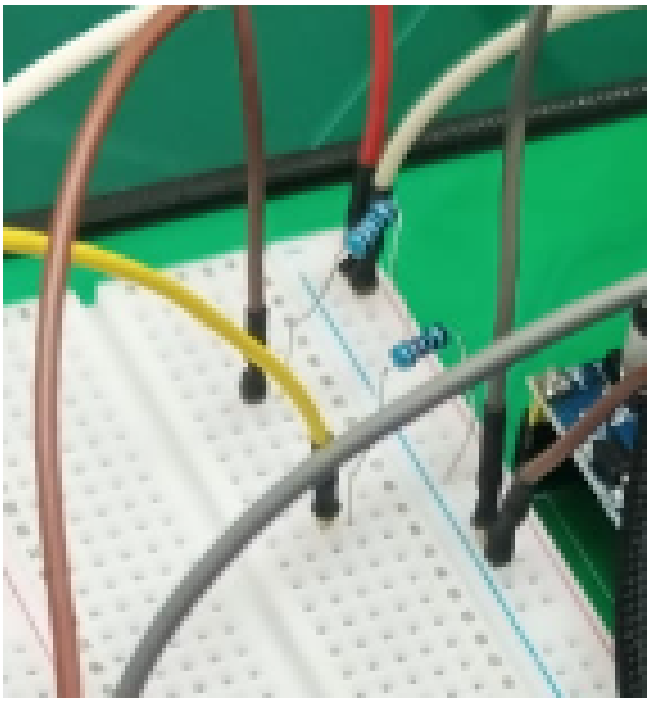
Gambar 19: STM32 Blackpill dan Jumper FTF



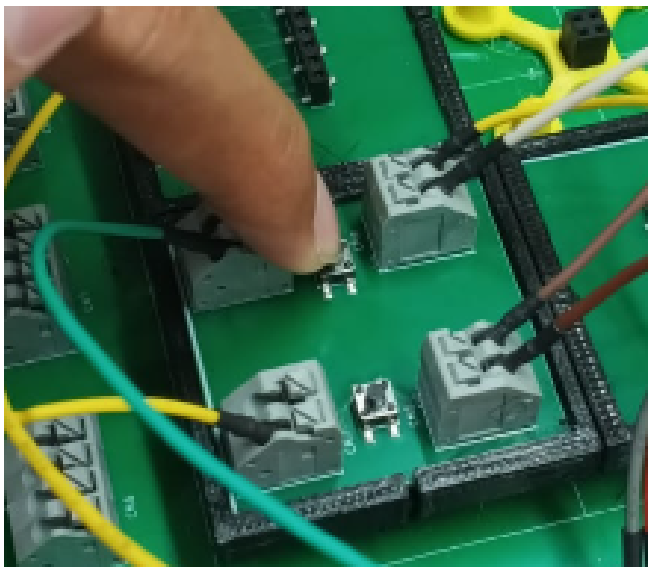
Gambar 20: ST-LINK



Gambar 21: ESP32 dan Kabel Type C



Gambar 22: Resistor 10k



Gambar 23: Button